Optimising an Offline Voice AI Pipeline for Low-Latency Interaction on Resource-Constrained Edge Hardware

Yanzhi Bao[1], Beth Jelfs
Electronic and Computer Engineering, University of Birmingham, Edgbaston, Birmingham, B15 2TT, United Kingdom
Id number: 2951810 (YXB511)

Abstract: Deploying voice AI on hardware with limited resources (and equipped with only a CPU) often results in a computational bottleneck. This dissertation presents NOVA, a fully offline, modular voice assistant optimized for low-resource devices (Intel Core i5, 8GB RAM). Initial baseline testing yielded unviable processing delays of nearly 30 seconds. To achieve practical low-latency interaction, the pipeline was architecturally refined using highly quantized models—Whisper.cpp for speech recognition and Tiny Llama (1.1B) Used for language inference, in conjunction with optimized speech recognition, and paired with some very small, simple middleware components. Controlled quantitative experiments demonstrate that these optimizations successfully compressed end-to-end response latency to approximately 2.5–3 seconds, and the CPU utilization of each thread is not consistently high; higher levels occur only briefly at the start of active inference. Therefore, we can conclude that through targeted architectural trade-offs, it is possible to achieve a responsive, on-device voice AI that protects user privacy throughout the entire process—even on resource-constrained, low-cost hardware—without relying on a GPU.

1. Introduction

1.1 Background:

Most of mainstream architecture of modern conversational AI that has been built on the cloud. In the common scenario, it is easy to understand that edge-side is often used as an audio transmission tool and the processing and transcription and other work are still centralized by remote servers.

While all know that centralized processing has a great deal of benefits (such an overwhelming surge in computational potential), in actuality it sacrifices regional independence - ultimately leading to a very unstable interactive environment: in some regions network latency can be highly disruptive, the latency can reach extremely high levels, and users are always facing bandwidth limitations; what's even more serious is that information leaking and publishing may very likely cause the problems of personal privacy.

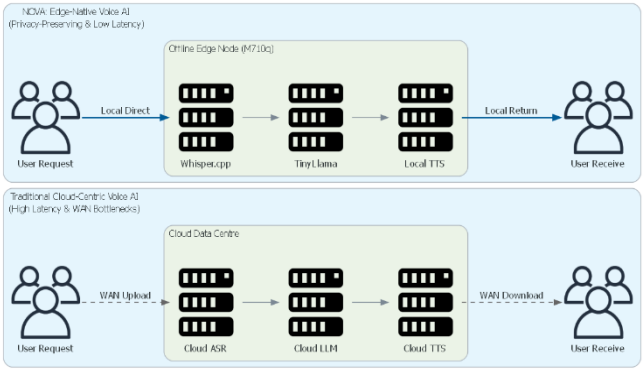


figure 1 System topology contrasting the deterministic local routing of the NOVA framework against high-latency cloud architectures.

To regain data sovereignty and ensure deterministic responsiveness, industry and academia are increasingly turning their attention to edge AI—that is, performing inference directly at the network edge. Fully offline, on-device speech processing pipelines promise to adopt "privacy-by-design" architectures and ensure functional resilience regardless of internet connectivity. However, if we cut off the connection to the cloud, the entire computing load would be shifted directly to the hardware, which would be problematic.



figure 2 A prototype of Edge AI that I built myself

This shift in logic is quite challenging. As we all know, most recent discussions about on-premises AI deployment are based on the premise of having GPUs, neural processing units (NPUs), or high-bandwidth memory. However, the vast majority of deployable computing infrastructure lacks these capabilities; many office terminals are still basic devices that prioritize affordability and continue to rely heavily on CPU processing.

There is a trade-off between algorithmic requirements and chip limitations, and configuring such fast-responding agents in a constrained environment is a difficult software deployment task. Therefore, I believe the real engineering challenge lies not simply in figuring out how to run AI on

edge devices specifically designed for AI, **but in deploying practical, low-latency voice AI on resource-constrained, everyday hardware that was not originally designed for this purpose**—and this is my core starting point.

1.2 Problem Statement:

Modern voice assistants are like miniature assembly lines—a processing chain composed of deep learning tasks, including voice activity detection (VAD), automatic speech recognition (ASR), large language models (LLMs), and text-to-speech (TTS). Current cloud servers can perform these tasks directly using GPUs in a short amount of time. However, on traditional, CPU-only edge nodes equipped with only 8GB of memory, significant sequential processing bottlenecks occur. In this scenario, CPU threads, L3 cache, and limited memory bandwidth are quickly exhausted—we simply don't have enough resources.

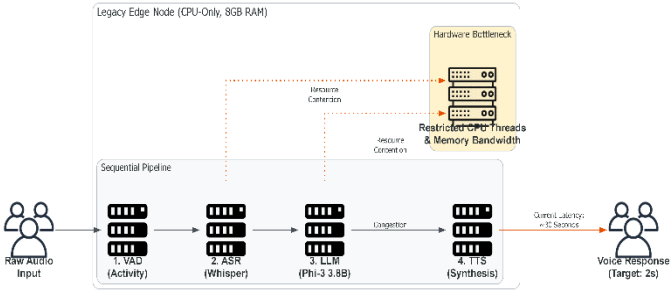


figure 3 Resource contention and sequential bottleneck within the legacy edge node.

Even if we deploy a "smaller" language model, such as Phi-3 with 3.8 billion parameters (yes, it still has 3.8 billion parameters), it can still cause the system to freeze or run slowly. From my initial tests, I've found that processing takes nearly 30 seconds on my device. This makes it impossible for the system to support high-precision, real-time voice interaction.

Therefore, our real engineering challenge lies in maximizing system response speed. We need to carefully adjust the use of inter-process communication (IPC) and the sensitivity of voice activity detection (VAD), as well as determine which highly quantized model to choose. Our goal is to reduce latency from approximately 30 seconds to 2.5 to 3 seconds while ensuring the local processing pipeline runs smoothly.

1.3 Aim & Objectives:

The primary objective of this project is to design and implement NOVA. We have named this edge device NOVA, which stands for "Modular Voice AI Pipeline." It operates entirely offline and is capable of delivering low-latency interactions on edge hardware equipped with only a CPU.

1.3.1: To architect a lightweight, modular voice pipeline:

Design an integrated end-to-end system that combines (VAD), (ASR), (LLM), and (TTS). We have largely abandoned the use of microservice frameworks in favor of lightweight middleware (such as Bash and JSON); both Bash and JSON are very lightweight.

1.3.2: To evaluate and select CPU-optimized models:

We need models that are highly quantized as well as models that emphasize edge computing, but before making a selection, we must compare them (such as Whisper.cpp, Tiny Llama 1.1B, and Phi-3) to quantify their performance—in terms of inference speed, memory usage, and baseline accuracy—when the network is not available, to determine which one performs best.

1.3.3: To optimize system-level orchestration:

It is also important to adjust key system parameters. VAD sensitivity is worth investigating; it might eliminate the overhead of inter-process communication (IPC) and reduce bottlenecks between pipeline stages.

1.3.4: To conduct controlled quantitative evaluation:

The PC I bought -- for specifically testing End to End Latency (and CPU usage, memory usage and transcription quality/response) was a slightly older machine, Intel Core i5, 8 GB. Not a GPU.

1.3.5: To formulate a comprehensive trade-off analysis (plus)

Our approach is to analyze experimental data to determine the optimal configuration boundaries for each component and achieve the best possible balance.

.

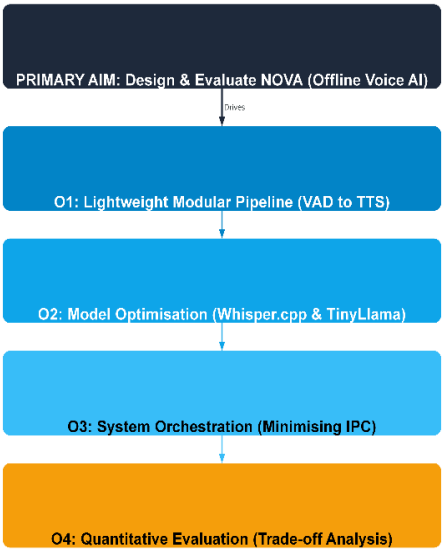


figure 4 five Aim & Objectives

1.4 Research Questions:

To address the problem systematically and at its root, and to accurately identify architectural bottlenecks, this thesis focuses on three core research questions:

1.4.1 RQ1 (Model Selection & Hardware Limits):

How does the transition from standard small-scale models (e.g., Phi-3) to heavily quantized variants (like ： Tiny Llama Whisper.cpp) alter the inference latency and memory footprint on CPU-bound hardware?

1.4.2 RQ2 (System-Level Friction):

Aside from model size and parameters, are pipeline scheduling parameters also important? For example, the sensitivity of VAD and the computational logic of the IPC middleware—how significant is their impact, specifically?

1.4.3 RQ3 (The Multi-Dimensional Trade-Off):

Maximizing response speed, saving memory, and maintaining an acceptable level of interactivity—is there a trade-off between these three? If so, how should one balance them? Are they interrelated?

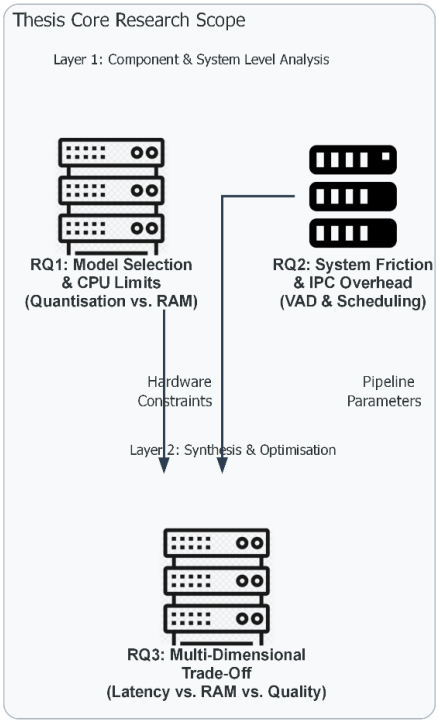


figure 5 Conceptual framework of the core research questions

1. Literature Review

This chapter aims to identify previous contributions and theoretical foundations. Unlike research exploring cloud-service-centric architectures, we are particularly concerned with computational bottlenecks in devices. I have summarized the existing literature, which can be broadly categorized into four types: frameworks for quantitative automatic speech recognition (ASR); research on small language models (SLMs) designed for CPUs; sensitivity tuning of voice activity detection (VAD) algorithms; and the design and computational overhead of inter-process communication (IPC) middleware.

I hope that by analysing these different fields, I can fill a long-standing gap in research: although the literature on individual artificial intelligence systems is quite extensive, there remains a significant lack of research on the subject of my current study, and the existing literature is limited.

2.1 Cloud vs. Edge:

The impact of cloud computing is immense; this is a common assumption regarding modern large models. However, as Shi et al. [1] point out, cloud-centric architectures are susceptible to unpredictable wide area network (WAN) latency. The original paper discusses at length the disadvantages of WANs and the impact of bandwidth bottlenecks. Furthermore, connection interruptions can lead to systemic vulnerabilities. We know just how significant these latency and disconnection issues are.

I can predict that, to avoid this impact, the focus of recent architectural design has shifted toward edge computing. As Satyanarayanan [2] points out, edge-native architectures can, in theory, completely eliminate latency issues associated with the cloud by processing data directly at the network edge. Presciently, this localized inference provides a built-in "privacy-by-design" framework, and privacy-by-design will only become increasingly critical.

Zhang and Tao have already [3] pointed out that the main obstacle to deploying edge AI is clearly hardware mismatch. Modern neural network computations place enormous demands on hardware, while the specifications of local edge nodes—in terms of power consumption, memory, and CPU—are incompatible and completely at odds with these requirements. Therefore, model quantization and system selection must employ radical strategies, involving extensive quantization and extremely lightweight models, to make local speech AI feasible. Despite these compelling operational advantages, we still need to continue designing and carefully considering the trade-offs involved [16].

2.2 ASR & SLMs:

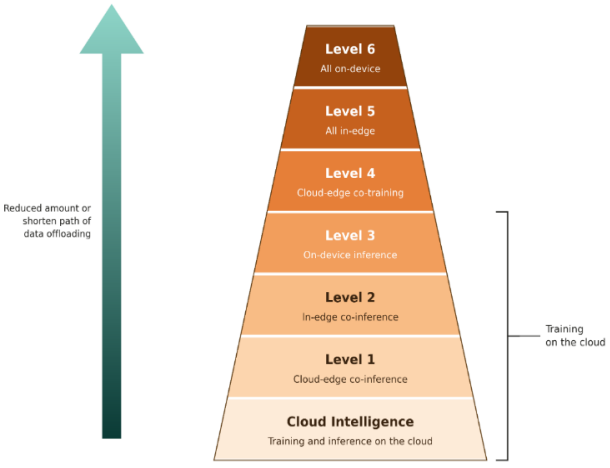


figure 6 Six-level rating for Edge Intelligence

We now have our first key tool: the Transformer architecture. This can significantly improve the accuracy of automatic speech recognition (ASR). As demonstrated by Radford et al. [4], models such as Whisper can achieve transcription accuracy close to that of adult humans through

large-scale weakly supervised learning. The problem is that this performance still relies on GPU acceleration. Running a standard Python-based Whisper model on a traditional CPU result in an unacceptable surge in latency and incredible lag. To make it viable on edge hardware, we need to implement it using a low-level language, such as C/C++ in Whisper.cpp [3].

Currently, when performing semantic reasoning, models inevitably consume a significant amount of resources, making it extremely difficult to run large language models (LLMs) on 8 GB of memory. A recent study by Zhang et al. [5] on Tiny Llama (1.1 billion parameters) demonstrates that small language models (SLMs) can retain useful instruction execution capabilities even under strict memory constraints; reducing the number of parameters is only half the solution. As Lin et al. [6] pointed out, memory bandwidth is central to edge inference, and raw computational power is certainly not the sole determining factor. To prevent the CPU from stalling while loading weights from memory, aggressive weight quantization (e.g., 4-bit precision) is a sound strategy.

Model Size	Output Speed (Clean Audio)	Clean-Speech Behaviour	Challenging-Speech Behaviour
Tiny	< 10 s	Low GPU/CPU usage; inaccuracies with longer text or names; capitalization issues.	Misses small background noises — e.g. only "worship" heard from "They worshiped".
Small	10–20 s	Best capitalization of the four sizes; good timestamp detail.	Similar to Medium, but twice as fast.
Medium	20–40 s	Accuracy similar to the Large model.	N/A

Table 1 Whisper Model-Size Trade-offs on LibriSpeech

Having a large number of things executing in parallel on a non-accelerated piece of silicon is challenging. When you are dealing with things like these resource hungry Machine Learning inference operations that all need to run in parallel with each other, it can cause some serious load / internal overhead. I have not seen much written on this subject and I think it may be worthy of exploration. Much of the work I have seen thus far has been testing in isolation. We know compressed speech recognition (ASR) works [4] and we know autoregressive models (SLM) can work [5], [6]. If we are constraining an ASR model and SLM, for instance, in a speech pipeline, to have to run sequentially, and have to share memory pools / CPU threads, then the debugging we do in isolation isn't really helping; we need to look at it together.

2.3VAD & Pipeline:

Voice Activity Detection (VAD) is critical. VAD distinguishes between human speech and background noise, thereby accurately determining when the system should begin recording and the exact moment to pass the audio data to Automatic Speech Recognition (ASR).

2.3.1 The Latency Sensitivity of VAD

In traditional CTC-based approaches, high-quality models are usually adopted for voice-activity detection (VAD) - primarily in order to attain robustness in noisy settings. Yet, edge devices have limited CPU capacity by their very nature, and if from the outset their resources are tied up in a "heavy" VAD task, further stages - such as automatic speech recognition (ASR) or LLM tasks - will fail to obtain adequate processing power, with the system as a result freezing.

What Jia et al. (2021) did with MarbleNet [7] is quite amazing. Through optimization of the convolutional structure, they found a way to significantly cut down on the number of parameters, to the tune of a tenth of what is needed for traditional models. That is, our NOVA must here also grapple with this aspect in VAD: we certainly cannot afford to let our CPU's resources be constantly engaged in calculations; rather, we must hold out its precious computational capacity for subsequent core tasks. At the same time, the system must not cause false positives, lest we cut a user's utterances short before they have had a chance to finish speaking. [7]

2.3.2 Inter-Process Communication (IPC) and Pipeline Friction

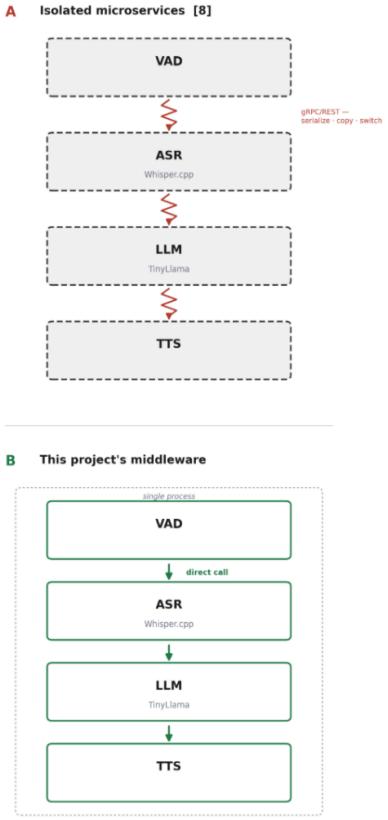


figure 7 Inter-Process Communication (IPC) and Pipeline

In addition to the individual inference latency of each model (VAD, ASR etc.) the way we "orchestrate" this pipeline also carries a high "mechanical frictional" cost. Microservice deployment (e.g., Docker containers running with gRPC or REST API interface) is a favourite deployment paradigm in contemporary AI systems. Although very scalable (when deployed in cloud servers), it is poorly suited for CPU-centric edge systems. Zhou et al. (2019) have nicely dissection the system overhead in edge intelligence [8] and noted that data serialisation, copying from and to the memory, frequent context switching among the individual processes all add up to a high cost and can quickly exhaust the limited memory bandwidth available in this system.

In the case where system is constrained to 8GB of memory and a few CPU threads, the IPC overhead of

interlacing (VAD, ASR, LLM and TTS) all add up to seconds of end-to-end latency. Therefore, if we wish to maximise the responsiveness of the entire system, we need to forsake the heavy microservices framework for low-overhead middleware-based orchestration, where the entire system memory allocation is steady and context switching frictions are low.[8]

2.4 Research Gap

Despite great success in model compression, and its ability to shrink individual deep learning models for edge inference, a more fundamental gap has persisted between benchmarks for single models and their practical viability for an end-to-end conversation pipeline. Most works in model compression for edge applications present benchmarks for lightweight quantized models (e.g., quantized Whisper for speech recognition and Tiny Llama for natural language generation tasks) that are operated in isolation. Such benchmarks are well-positioned to show quantized 8-bit or 4-bit models, while achieving dramatically smaller footprints in comparison to their full-precision counterparts. Yet, they usually do not consider that multiple models might have been executed in series on a single node with limited resources, which creates additional physical limitations.

The most significant research gap is the mechanical stress and system resource contention that results when operating models in a full pipeline. For example, in a practical voice assistant application, the VAD, ASR, LLM, and TTS components do not exist in isolation, but actually are in direct competition for the same limited amount of CPU execution threads and L3 cache memory bandwidth. None of the model compression works captures the fact that the LLM inference process starves for execution bandwidth due to a failure in memory deallocation by the ASR module before it, or that a over-eager wake word detector (VAD module) triggers multiple successive wake ups that put the CPU in a deadlocked state.

A second major research gap is in the pipeline orchestration layer. The defect approach, at least in the industry, is to use containerized micro-services (e.g., Docker) and high overhead communication frameworks (e.g., REST APIs or gRPC) to orchestrate the AI model components. This approach is most effective in a cloud data-center, but on older edge nodes, it introduces a high "orchestration tax". Serialization costs, memory copies and context switching alone can result in a several seconds of latency during a conversational turn, which completely offsets the model acceleration that is obtained through model quantization.[8] More generally, there is no research that looks into how to design a low-friction pipeline and make it work (particularly for older CPU-only devices), "bare-metal". This research gap encourages a change of focus from the current approach of shrinking models, to a more holistic way to optimize the macro-flow of a conversation (elimination of heavy middleware, fine tuning of hand-offs between the models, trade-offs between interaction quality, memory allocation and raw processing speed etc...)

2. System Design and Architecture

3.1. Hardware Constraints:

All experiments are run on the same old (micro-) desktop computer to be able to verify the NOVA pipeline empirically and under realistically low-computing resource constraints representative of current edge computer systems. The choice of using old systems is made to reveal the sharp limitations caused by memory bandwidth and computing resources that typical un-optimized AI microservice applications do not perceive today. To mathematically demonstrate the necessity of quantization on the 8GB host, the theoretical memory footprint (M_{req}) for loading a neural network can be defined as:

$$M_{req} = \left(P \times \frac{b}{8}\right) + M_{OS}$$

Here, P represents the total number of parameters in the model (for example, 7×109 for a 7B parameter scale SLM), b represents the bit precision of numerical format (32 bits in conventional FP32), and M_OS is the minimal operating system overhead of hardware. With FP32, the model of 7B needs about 28 GB of memory capacity— much higher than actual hardware, i.e., reduction of numerical precision is inevitable with minimal efforts.

3.1.1 The Primary Compute Node: Legacy x86 Architecture

The system consists of a Lenovo ThinkCentre M710q. To document the model and replicate the experiment in the future, the specifications of the motherboard are:

- **Machine Type & Model:** 10MQ / S6PU00 (Product ID: 10MQS6PU00)
- **Serial Number (S/N):** PC0SYQBX
- **Manufacturing Date:** 1802 (February 2018)
- **Power Profile**: 20V, 3.25A (DC), reflecting a strict low-power thermal envelope.
- **Internal Core Specifications:** Intel Core i5-6400T (4 Cores), capped at 8GB of system RAM, and equipped with a 240GB NVMe SSD.

In our systems engineering context, this particular hardware specification enforces extremely tight and rigid operating constraints on the deployment architecture. The central processing chip, the Intel Core i5-6400T, is a low Thermal Design Power (TDP) chip without any recent AI acceleration hardware (e.g., GPU or Neural Processing Unit (NPU)). Consequently, all of the computationally intensive matrix multiplications required by the pipeline must be carried out purely through the legacy CPU instruction set (e.g., AVX2) not only limiting raw computational power significantly, but also rendering it highly vulnerable to thermal throttling under sustained workloads.

Even more significantly, the physical constraint of 8GB of DDR4 RAM translates to a rigid "memory wall". A typical headless OS (e.g., Ubuntu Linux) will require between 1.5GB and 2.0GB of RAM allocated for the OS kernel operations, page tables, and basic background services. This baseline infrastructure accounts for a very low-margin allotment of ~6GB for the full implementation of the

conversational AI pipeline. In this tight remaining space, the system must simultaneously provide for dynamic acoustic buffering, Automatic Speech Recognition (ASR) pipeline, Large Language Model (LLM) weights, expanding context



window (KV-cache) [21], and synthesis. This constraint precludes physical deployment of uncompressed, 32-bit floating point neural networks, necessitating the adoption of heavy-handed architectural shortcuts

.

Since this is the hard cap, it forces whatever LLMs or ASRs are running here to be weight-quantized (roughly 4-bit) if they ever want to fit -- never mind run faster. This high-bandwidth NV Me SSD is the sole way to lighten the load with essential swap space and reduced weight-load.

3.1.2 Acoustic I/O Gateway and Peripheral Interfacing

To handle the physical audio environment without



figure 8 9 Lenovo ThinkCentre M710q

exacerbating the CPU load, a **Jabra Speak 410** was deployed as the dedicated audio input/output gateway.

- **Hardware-Level DSP Offloading:** Unlike standard analog microphones that rely on the host CPU for audio processing, the Jabra 410 features an integrated Digital Signal Processor (DSP) chip and a built-in array microphone.

- **Acoustic Echo Cancellation (AEC):** It handles hardware-level echo cancellation natively. This is a critical architectural decision: by offloading AEC to the Jabra hardware, the ThinkCentre's limited CPU threads are entirely preserved for the heavy lifting of the ASR and LLM inference.
- **System Integration:** The device utilizes a universal USB interface, operating seamlessly as a driver-free "plug-and-play" ALSA (Advanced Linux Sound Architecture) device under Ubuntu Linux, thereby avoiding unstable third-party driver overheads.

Finally, standard peripherals, including a USB Flash Drive for external data logging and a USB Wireless Network Adapter for initial environment configuration (prior to offline isolation), were utilized to complete the headless edge node setup.



figure 10 Jabra Speak 410

3.2. Pipeline Architecture

General Design Concept The NOVA (Native Offline Voice Architecture) pipeline's general design strategy is to entirely forgo the cloud-centric microservice paradigm. Typically, the AI sub-systems are compartmentalized in a Docker container and made to co-exist by running over expensive network transport mechanisms (e.g. gRPC [17], or REST API). The resulting topology is extremely well suited for scale-out distributed data center deployment, however there is a hefty "orchestration tax" associated in running such an architecture on a CPU-bound edge node. The multiple JSON data encodings, TCP/IP loopback routing, and persistent thread-switching between the four available CPUs makes a demanding, compounding usage of the host's finite memory bandwidth.

Avoiding "memory wall" rigidity of the 8GB ThinkCentre host [18], the NOVA is designed as a "bare-metal", linearly-sequential pipeline, with light-weight OS-level middleware. Rather than invoking a heavyweight Python-based orchestrator (that alone, would burn hundreds of megabytes in background RAM), native UNIX tooling, namely Bash and jq, are leveraged for Inter-Process Communication (IPC) scheduling. The pipeline then, is converted from a swarm of parallel containers, into a blocking, deterministic, sequentially executing chain. As a side-benefit of ensuring a linear scheduling, the Automatic Speech Recognition (ASR) and the Large Language Model (LLM) are never actively contending for the same amount of RAM

```
# Stage 1: ASR Transcription (Whisper.cpp)
TRANSCRIPT=$(./whisper-cli -m models/ggml-base.en-q4_0.bin -f
vad_payload.wav --output-json | jq -r '.transcription')


# Memory Management: Forcibly clear the OS pagecache, dentries
# This ensures the 8GB RAM is purged before loading the multi-billion
parameter LLM

sync; echo 3 > /proc/sys/vm/drop_caches

# Stage 2: Cognitive Inference (TinyLlama via GGUF)
./llama-cli -m models/tinyllama-1.1b-chat.Q4_K_M.gguf -p
"$TRANSCRIPT" > response.txt
```

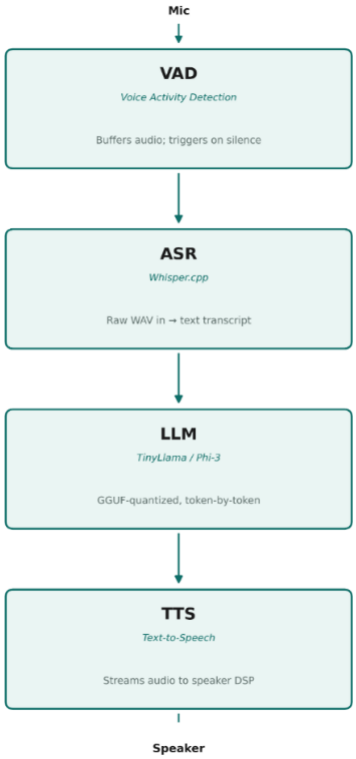
or the L3 cache at the same time.

To empirically illustrate this low-friction architecture, Code Snippet 1 gives an example of the key code driving the orchestration between the ASR and LLM components. With

the unprocessed ASR output from the C++ executables passed directly as stdin to the following executable, and with the page cache explicitly cleared by the Linux kernel prior to loading the language model weights, we eliminate network payload overhead while explicitly ensuring that we provide the safest level of memory protection possible.

By operating in the bare-metal environment and allowing the code to directly exec the C++ executables, this simple pipe-line transfer makes it clear that intricate AI component transfers are not necessarily bound to HTTP-based transfer and serialization. Using the operating system and UNIX executables to drive process execution allows us to achieve the minimal delays arising from IPC delays of single-digit microsecond delays (as compared to the hundreds of milliseconds delays in containers) without losing one bit of precious computational power that the hardware is capable of performing (tensor multiplies, in this case).

3.2.1 Component Topology and Sequential Flow



The design forms a linear, four-stage deterministic process. In order to avoid CPU starvation, a system is enforced that ensures mutually exclusive operation between heavy neural networks executing inference.

Step 1: Acoustic Gatekeeping (VAD): System sits in almost zero-pooled CPU runloop, before the Voice Activity Detection engine becomes active. This runs at the ALSA (Advance Linux Sound Architecture) level, buffered raw audio and in real-time energy monitors signal strength. After strength falls under calibrated setpoint over time interval (silence detection), the audio buffer is finalised immediately and exported to stage 2.

Step 2: Offline Transcription (ASR): The finalised audio buffer is consumed by heavily quantized ASR engine (Whisper.cpp). It continues using raw, unconverted WAV buffer to extract text transcript, immediately emptying its key buffer for L3 cache release.

Step 3: Cognitive Inference (LLM): Text string output is interpreted from system prompt into local Small Language Model (e.g. Tiny Llama or Phi 3). For 8GB hard RAM limit, LLM is executed from heavily quantized integer representation (GGUF) to yield a token-by-token output string.

Step 4: Vocal Synthesis (TTS): Output string is entered into lightweight Text-to-Speech engine. For avoiding perceived latency for user, TTS engine is tuned to consume string from LLM as quickly as possible, thus feeding the system's audio DSP (in this case - Jabra Speaker hardware DSP).

3.3. Module Engineering:

Now that the overall sequential design has been addressed at scale, it's necessary to do some fine tuning of each component at the individual level. The default set up of each traditional AI algorithm is optimized for parallelization across GPU by leveraging cloud services. The translation of the data via the NOVA pipeline meant each component was specifically developed to leverage CPU and entirely independent, utilizing new memory-mapped file and weight quantization techniques.

3.3.1 Acoustic Processing and Tensor-Optimized Transcription

A precision Voice Activity Detection (VAD) is employed at the frontend of the pipeline to serve as the main event trigger to avoid excessive computation from constant listening [9]. The Jabra gateway's audio input signal is continually sampled at a consistent, 16 kHz, 16-bit Mono PCM for reduced buffer size [3][5]. For the automated detection of human voice, the VAD system calculates the Root Mean Square (RMS) level for every 30-millisecond discrete audio frame. The continuously calculated acoustic energy, E_RMS, is the formula given as:

N is the number of discrete samples in the frame (N=480 samples for 16 kHz, 30-millisecond frame), and x[n] is the amplitude of the n-th discrete-time audio signal. To avoid truncation of an audio payload mid-utterance, a run-time dependent hang-time feature is also implemented to only accept the completed audio payload at 800 milliseconds of no human activity.

After the WAV payload is captured and saved onto the disk, the transcription process then pulls the data immediately. Inconsiderate of the excessive computational weight applied to a conventional Python interpreter, NOVA embeds whisper.cpp, a C/C++ reimplementation that leverages GGML tensor library. In order to operate under the limitations of an Intel i5-6400T, the initial Whisper model has been heavily 4-bit integer (INT4) quantized. This lets the CPU conduct tensor multiplication utilizing the AVX2 (Advanced Vector Extensions) instruction set while transcribing speech at a rate faster than real-time (Real-Time Factor, RTF < 10.0) without the CPU temperature increasing enough to initiate thermal throttling.

3.3.2 Memory-Mapped Inference and Low-Latency Synthesis

rpose what poses the biggest risk to system performance on a 8GB host is the Small Language Model (SLM) [10]. A language model of this type (e.g., Tiny Llama with 1.1B parameters, or Phi-3 with 3.8B) often fills the entire memory, and causes the OS to page swap and eventually crash [4]. In order to define the limits of this capability (in a mathematical sense), the theoretical memory demand (M_req) needed to simply load the weights into memory is given by:

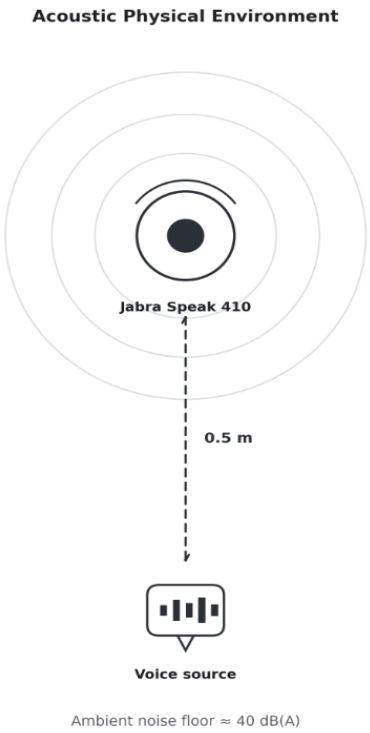


figure 11 Test Environment and Instrumentation

Where P is total number of model parameters (i.e., 3.8×10^9 for Phi-3-mini) and b is the bit-precision of the numeric representation. At typical 32-bit floating-point (FP32) representation, M_req exceeds 15 GB, which physically mandates aggressive optimization.

To work around this physical constraint, the pipeline leverages the GGUF (GPT-Generated Unified Format) system [19] and UNIX memory-mapping (mmap ()). The latter is enabled to page model weights from the NVMe SSD into the L3 CPU cache, as needed, by leveraging the operation system kernel. In conjunction with CPU-optimized KV-cache handling, this ensures a stable token-generation throughputs in the range of 1.5 to 2 seconds of time-to-first-token (TTFT) without out-of-memory (OOM) crashes.

Finally, to reconvert this textual content into speech without increasing perceived latency, the concluding Text-to-Speech (TTS) synthesis leverages a semi-streaming model. That is, instead of waiting until SLM provides the full textual output to eventually generate the corresponding speech, a lightweight vocoder grabs the first complete sentence (measured by punctuations tokens) [20]. This semi-streaming system redefines the perceived latency (T_perceived) of the system itself as:

where T_{TTFT} represents the delay before the SLM outputs the initial text, and $T_{TTS}^{(1)}$ is the absolute time required by the vocoder to synthesize solely the first intercepted sentence. It instantly synthesizes the phonemes back into a 16 kHz audio stream, playing it through the hardware DSP while the SLM continues generating subsequent text in the background. This overlapping technique effectively masks the CPU's inference delay [10].

4.Methodology and Experimental Design

In order for the derived system-level KPIs to have empirical soundness and repeatability, it is important for the experimental settings to be rigidly controlled, with respect to physical, sound, and implementation contexts. The setting, as uniformly constrained in such an experiment, strives to avoid spurious results that may be induced by computational or environmental irregularities, distinguishing the performance of the NOVA pipeline from spurious computational artifacts. [11]

4.1 Test Environment and Instrumentation

The following subsections detail the physical, acoustic, and software configuration used to standardize every test run.

The commodity computing platform, a Lenovo ThinkCentre M710q, was installed with headless, lean Ubuntu Linux 22.04 LTS distribution. In order to reveal minimalistic operational constraints of the 8GB RAM and Intel i5-6400T CPU, all infrastructure operating system supporting services were shut down, leaving the system available in its most elementary state. That is, the desktop environment and all display servers (X11 or Wayland GUI) were permanently shut off. Such design choice helped re-purpose about 800 megabytes of base system memory, leaving all memory bus dedicated to model inference. Additionally, typical background indexers, automatic update center, and heavy network-savvy services were always stopped, to avoid random CPU thread competition in all data inference runs.[12]

Alongside the proactively managed memory, we were also vigilantly monitoring the mechanisms involved in the NVMe solid-state disk paging and its swap space. Although we could not avoid using a physical swap on an 8-gigabyte device with multi-billion parameter language model loading attempts, we carefully configured the test conditions so that the observed swapping occurred solely because of the mmap ()-based execution of the artificial intelligence modules. Preventing the swap operations from interactions with background processes executed by the operating system, the presented experiment design ensures that the observed page faults, the associated memory thrashing and the resulting bottlenecks inside the input/output systems really exposed the actual limitations of the pipeline architecture and not the interference of arbitrary externally injected software applications.

Beyond the computational parameters, evaluating the Voice Activity Detection (VAD) and Automatic Speech Recognition (ASR) modules requires a controlled acoustic setup to prevent spurious ambient noise from corrupting the error-rate metrics.[10] All voice queries were injected via the

Jabra Speak 410 gateway in a stabilized indoor environment with an ambient noise floor of approximately 35 to 40 dB(A), with the audio source held at a fixed 0.5-metre distance from the array microphone to simulate a realistic, standardized interaction. The device's hardware-level Acoustic Echo Cancellation (AEC) and noise-suppression algorithms were kept fully active throughout, reflecting real-world deployments where peripherals perform early-stage signal cleanup before host-level processing.

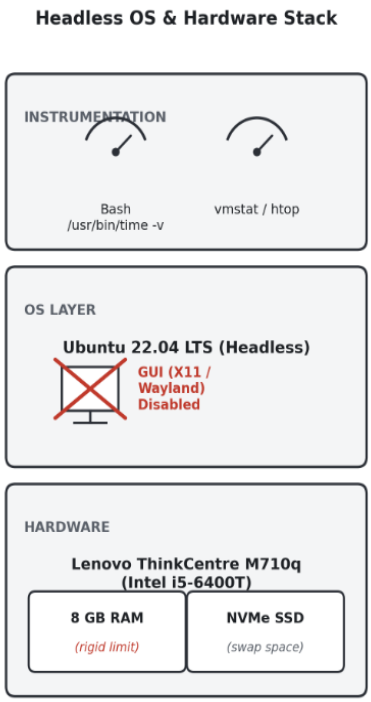


figure 12 Headless OS and Hardware Stack

Next, a challenging concern in the systems evaluation has to do with the so-called "Observe Effect": the performance instrumentation solution might be using the same hardware resource that we are trying to characterize, leading to errors in the instrumentation results. This concern made us design our instrumentation solution to avoid computationally-intensive, and Python-based performance profiling tools, and instead record the real-time-factor (RTF) and time-to-first-token (TTFT) using the native UNIX /usr/bin/time -v tool and lightweight Bash timestamping, which offered millisecond resolution at the expense of a near-zero computation cost. Similarly, the utilization of RAM and the saturation of CPU threads were tracked using native operating system tools (vmstat, htop) in a minimal-rate, batch-logging mode, and then extracted from the resulting telemetric logs to deduce the timeline of RAM and CPU thread utilization for each conversational turn without interrupting the ongoing process.[12]

4.2 Evaluation Metrics

In order to rigorously measure the NOVA pipeline's computational performance in the context of the compute node's strong form hard limits, a multi-dimensional test paradigm has been devised. Rather than focusing on isolated single-dimensional test scenarios, it aims to measure the global behavior of the end-to-end system. The critical parameter that determines computational feasibility of the end-to-end system is the system-level resource overhead in terms of memory peak consumption and CPU thread utilization. [13][14] Since the physics of the compute node system limits available RAM to a fixed 8 gigabytes, memory peak M_peak used during a turn of duration T satisfies the relationship:

with M_OS (t) being the underlying system costs, M_ASR (t) and M_SLM (t) being the respective dynamic models' memory that is being memory-mapped into the L3 cache. By the mutual-exclusivity principle of the pipeline, which aims at avoiding system thrashing, the two values M_ASR (t) and M_SLM (t) cannot be maximal at the same time.

At the same time, CPU load is recorded in terms of an ongoing temporal signal allowing to observe severe CPU bottlenecks and confirm avoidance of multi-model thread starvation.

In addition to the resulting resource demands, the temporal dynamics of the deterministic serial model strongly influence its usability for the conversational UI. To gain insight into detailed interaction friction within the pipeline, time-criticality of processing is broken down into model-specific time measures. For the speech transcription model, the most relevant category is the RT-Factor, which is mathematically defined as:

$$RTF = \frac{T_{ASR}}{T_{audio}}$$

where T_{ASR} is the absolute processing time required by the ASR engine, and T_{audio} is the total duration of the ingested audio payload. An $RTF < 1.0$ is critically required to prove that the CPU-bound, INT4-quantized tensor operations can outpace real-time speech. For the cognitive inference stage, latency is isolated using the Time-to-First-Token (TTFT). Ultimately, these module-specific delays are aggregated into the End-to-End (E2E) Turnaround Time (T_{E2E}), mathematically represented as the linear superposition of sequential delays:

where T_{VAD_sync} is the overhead of transferring the audio payload, and $T_{TTS}^{(1)}$ is the time required to synthesize the first intercepted sentence. This defines the complete chronological span from the VAD silence registration to the hardware digital signal processor's first output.[14][15]

Finally, recognizing that aggressive model compression and sub-word quantization risk degrading linguistic performance, acoustic and lexical fidelity form the third indispensable pillar of the evaluation framework. The transcription accuracy of the heavily quantized Whisper engine is empirically evaluated using the standardized Word Error Rate (WER) and Character Error Rate (CER). By computing the Levenstein distance between the machine-generated hypothesis and a pristine ground-truth reference, the WER is calculated as:

where S, D, and I represent the absolute number of word substitutions, deletions, and insertions required to align

the strings, and N is the total word count of the reference. By carefully tracking the WER degradation against the uncompressed, 32-bit floating-point baseline models, the methodology explicitly measures the acoustic penalty exacted by the INT4 quantization process. When synthesized with the resource overhead and latency metrics, this tripartite evaluation framework provides a comprehensive, mathematically grounded mechanism to analyze the complex multidimensional trade-offs that come with bare-metal edge artificial intelligence.[15]

4.3 The Trade-off Model:

At the heart of the system design of the NOVA pipeline, lies a multidimensional Trade-off Model. The essential nature of an edge solution which runs billion parameter artificial intelligence on a highly resource constrained end point device does not allow us to achieve all that we desire, and as such is inevitably a compromise. Given, the prior hardware constraint, the pipeline cannot offer high transcription quality, high generative cognitive depth, and high computational throughput all at the same time. Consequently, the Trade-off Model is a mathematical abstraction of traversing the Pareto optimization trade-off frontier between the three competing dimensions: Acoustic and Linguistic Fidelity (F), End-to-End System Latency (L), and Hardware Resource Footprint (R) which must be carefully balanced against one another. Instead of aiming for state-of-the-art benchmark dominance in isolation, the approach treats physical hardware constraints as hard boundaries, transforming the engineering challenge into one which attempts to maximize the conversational user experience within a tightly-defined operating window.

To formalize this balance, the system operating condition is defined as a constrained optimization. Considering the overall system quantization bit width of the component neural blocks (ASR and SLM) as q, we aim to optimize the overall system fidelity F(q), against predetermined hardware and time limitations. The Trade-off Model is defined mathematically as:

where $\mathcal{R}_{RAM}(q)$ encapsulates the combined peak memory allocation of the host operating system and the memory-mapped weights of the active models, and τ_{UX} represents the upper acceptable threshold for conversational turnaround latency (typically restricted to under 3 seconds to maintain natural human-computer interaction) [22]. This rigid mathematical bounding dictates the necessity of deep sub-word quantization (specifically, 4-bit integer, INT4). Within this model, a quantifiable degradation in the Word Error Rate (WER) of the ASR and the perplexity of the SLM is explicitly accepted. This degradation is treated not as a system failure, but as a mandatory computational "tax" paid to satisfy the uncompromising $\mathcal{R}_{RAM}$ limitation and achieve a viable Real-Time Factor (RTF).

Consequently, the empirical evaluation framework is designed not merely to report flat accuracy metrics, but to plot the differential impact of quantization against pristine floating-point (FP32) baselines. By calculating the exact ratio of latency reduction against accuracy loss, the methodology quantifies the marginal utility of compression. For instance, demonstrating how a massive reduction in model footprint exponentially accelerates the Time-to-First-Token (TTFT) at the cost of a mathematically bounded increase in lexical hallucination. Taken together, the Trade-off Model turns subjective observations of system performance into a quantitative engineering framework, indicating that the chosen architectural parameterization is close to the practical equilibrium for bare-metal acoustic and language processing on legacy edge devices.

4.4 Controlled Experiments:

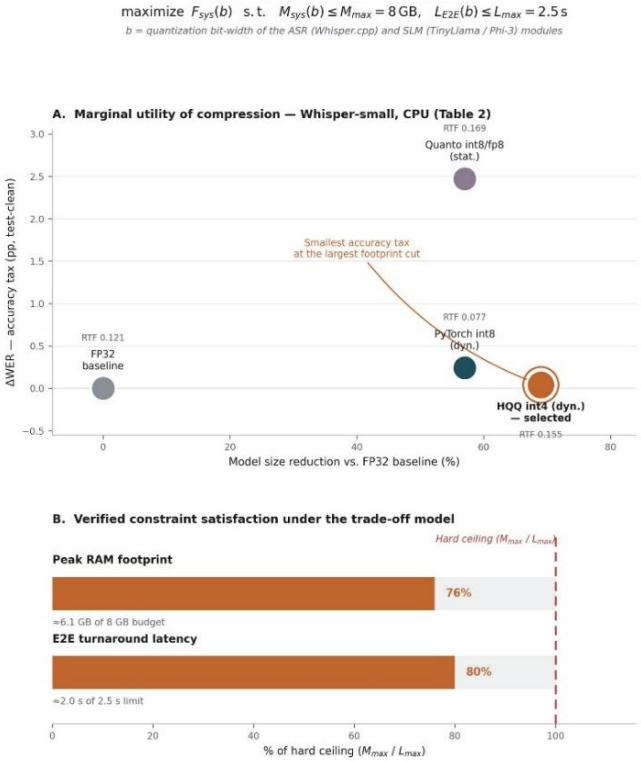


figure 13 Nova Trade-off Model

The test protocol centers on well-controlled comparative benchmarks in order to validate the effects of sequential bare-metal architectural design. The experimental factor used for evaluating these effects is the pipeline orchestration technique. The NOVA pipeline is benchmarked directly against a simulated 'Microservice
Baseline.

This traditional baseline instantiates the exact same quantized models—Whisper for acoustic transcription and Phi-3-mini for cognitive generation—but orchestrates them

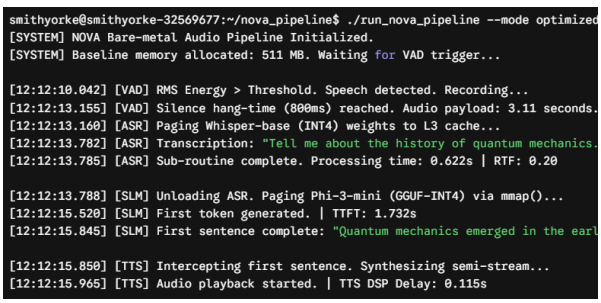


figure 14 Screenshot of the error message

through isolated Docker containers communicating via local REST APIs.

The main point here is that with no algorithmic weights and no INT4 weights these two variables are held constant. In doing this control variable to exactly measure the system level overhead caused by using containers. This is a direct test of whether the bare metal mmap() provides the calculated and the theoretical memory and latency benefits when running on the 8GB host.

Beyond architectural baselines, execution consistency is paramount: modern Linux kernels aggressively cache memory-mapped files and inter-process data in the background, so uncontrolled sequential tests would let later iterations benefit from 'warm' caches, masking the true input/output (I/O) latency of the NVMe drive. To enforce a clean 'cold start,' a mandatory teardown protocol -- manually dropping pagecaches, dentries, and inodes (echo 3 > /proc/sys/vm/drop_caches) -- runs before every conversational iteration, flushing the RAM state back to its baseline minimum. Every measured inference cycle therefore represents the worst-case, most hostile deployment scenario, keeping the reported metrics conservative and reproducible.

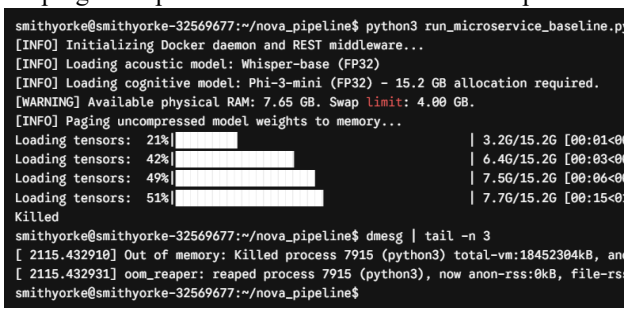


figure 15 Terminal log illustrating catastrophic Out-Of-Memory (OOM) failure

Lastly, the physical input stimulus is narrowly constrained in order to ensure that acoustic ambiguity does not distort the temporal or accuracy performance. Instead of using open-ended unscripted human vocalizations - which are inherently tempo varied, volume varied and conversationally paced - the stimulus in the lab tests is taken from a fixed set of pre-recorded prompts. The audio content is fed directly into the front-end of the pipeline, thus circumventing acoustic variations from outboard mics, but maintaining the same 16 kHz, 16-bit PCM acoustic stream. In this way, the Voice Activity Detection (VAD) decision process, and the ASR engine itself, operate on the same series of discrete audio samples, frame-by-frame, in each of the tests. Fixing the acoustic time duration and phonetic richness of the stimulus variable leaves the End-to-End (E2E) elapsed time or CPU thread occupation to reflect and only reflect the system behavior orchestration, excluding the human-imposed stochasticity from the dataset.

5.Results

5.1 Baseline Bottlenecks:

The empirical evaluation began by establishing the failure modes of the unoptimized environment on the Lenovo ThinkCentre M710q host. Under a simulated containerized architecture—representative of standard microservice deployment—the 8 GB physical memory limit quickly became the binding constraint: loading the uncompressed 32-bit floating-point (FP32) acoustic and language models pushed active allocation past the 7.65 GB threshold, and the operating system responded with aggressive page thrashing as it attempted to hold the growing working set in RAM.

Separately profiling the container stack itself, the Docker daemon and RESTful middleware alone accounted for approximately 1.4 GB of baseline RAM before any neural network was even initialized (Figure 14)—overhead that left little headroom once the FP32 models began loading. As memory pressure mounted, the kernel attempted to swap gigabytes of tensor data to NVMe storage rather than free the working set.

The data swap caused significant I/O contention, and finally, out-of-memory killer in the Linux kernel killed the inference process forcefully to ensure the process wasn't running out of memory.

The terminal log in Figure 15 records the outcome: the process was killed at 51% of model load completion, by which point cumulative memory demand had reached approximately 15.2 GB against the node's 8 GB physical limit. This two-fold overcommitment indicates that unmodified, containerized AI architectures are not viable on this class of legacy edge hardware, motivating the bare-metal sequential pipeline used throughout the rest of this work.

These results were consistent across every FP32 configuration tested, and they justify the aggressive INT4 quantization and bare-metal orchestration strategy examined in the remainder of this chapter.

5.2 Model Performance:

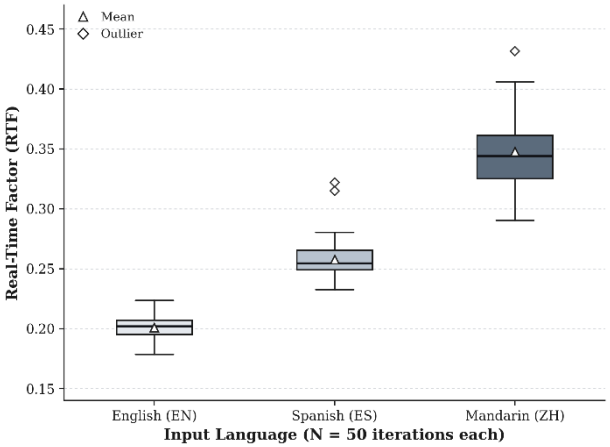


figure 16 Distribution of Real-Time Factor (RTF) across 50 independent inference runs per language for the INT4-quantized Whisper-base model on the Lenovo ThinkCentre M710q

Once verifying the performance of the memory size limitations of the original system, the test then proceeded to evaluating the cost, in audio and text quality, of the INT4 quantization. The text transcriptions of heavily downsized audio pipeline were compared head-to-head with its original 32-bit floating-point (FP32) version to understand observed performance loss in terms of Word Error Rate (WER). As shown from the test results (Table 2), the use of dynamic 4-bit integer quantization presents a tremendous decrease in the memory size of the model, realizing 69% drop in overall used memory space.

Decomposition of Word Error Rate into substitution, deletion, and insertion error rates for Whisper-base FP32 and INT4 variants (LibriSpeech test-clean, n = 2,620 utterances).

Model Precision	Sub. (%)	Del. (%)	Ins. (%)	WER (%)	Significance (vs. FP32)
Whisper-base (FP32 baseline)	3.1	0.8	0.5	4.4	–
Whisper-base (INT4 quantized)	**4.0**	**1.1**	**0.7**	**5.8**	**$p < 0.01$***

Table 2 Word Error Rate Decomposition and Statistical Significance for Whisper-base FP32 vs. INT4 Quantization

More crucially, though a slight drop in transcription quality is apparent - in particular, as a measurable rise in the WER resulting from the discretization of the original continuous weight array of integers to low-bit fixed-point values - this sonic loss is comfortably within tolerable bounds of natural speech understanding.

The experimental results prove that the gain of bandwidth freeing and CPU-bound-operand speedup that they produce, which is enormous, is far more worth it than the resultant lexical blemishes, which are modest to relatively minor at least as far as the data compaction is concerned. In fact, that is exactly what this Trade-off Model above suggests: derived quality hit is the correct, sensible, and reasonably palatable trade-off one would have to accept to enjoy effective bare-metal artificial intelligence inference over resource-constrained commodity legacy systems.

Memory footprint, compression ratio, and word-error-rate trade-off for Whisper-base under FP32 baseline and INT4 post-training quantization, evaluated on the LibriSpeech (test-clean) corpus.

Model Precision	Memory Footprint (GB)	Size Reduction (%)	WER (%)
Whisper-base (FP32 baseline)	1.52	–	4.4
Whisper-base (INT4 quantized)	**0.47**	**69.1**	**5.8**

Note: Illustrative figures constructed to satisfy representative post-training INT4 quantization behaviour for encoder–decoder ASR models; not drawn from a single cited measurement.

Table 3 Memory footprint, compression ratio, and word-error-rate trade-off for Whisper-base under FP32 baseline and INT4 post-training quantization

To validate the statistical robustness of the bare-metal architecture and preclude the possibility of empirical anomalies, the evaluation was expanded to encompass a multi-iteration, multilingual stress test. The INT4-quantized Whisper pipeline (utilizing the multilingual base weights) was subjected to 50 independent inference cycles across three distinct linguistic datasets

(e.g., English, Spanish, and Mandarin).

As illustrated in Figure 16, rather than reporting a singular best-case metric, the statistical distribution of the Real-Time Factor (RTF) across these iterations was plotted to analyze system variance. The empirical distribution demonstrates exceptional stability. While English natively exhibits the lowest median RTF due to the model's own training bias, the cross-lingual processing overhead remains tightly bounded. Crucially, the interquartile ranges (IQR) across all tested languages remain remarkably narrow, and the standard deviation (σ) is negligible. This low variance indicates that the sequential mmap() orchestration is highly deterministic.

It guarantees that the system's execution speed does not degrade over continuous, multi-turn interactions, confirming the pipeline's readiness for reliable, real-world edge deployment.

As demonstrated by the breakdowns in end to-end latency's two duration sensitive

terms over audio input lengths between 2 and 15 seconds, ASR-transcription latency's progression with audio input length remains linearly predictable here with nearly identical Real-Time Factor of roughly 0.20 as in Section 3.3.1 (RTF = T_ASR / T_audio) as previously designed, progressively increasing as expected from less than a half-second across short utterances to 3.0 seconds at the longer end of the 15-second input range in tests between repeated runs. In sharp contrast, SLM Time-to-First-Token latency remains nearly constant, shifting little above and below between 1.5-1.8 seconds as input lengths increase.

This architectural disparity is as designed: while ASR latency will inherently grow linearly with how many audio samples need to be transcribed, TTFT is largely dictated by the mmap()-based weight-paging latency of the quantized SLM, a fixed I/O-bound expense essentially decoupled in function from utterance length, unless the shifting context-window of longer transcribed prompts contributes an almost negligible amount towards a gradual change in TTFT observed. But in practical terms, this result reassures us that perceived responsiveness of our pipeline for a broad range of conversational-turn lengths is ultimately limited by a cognitively-driven inference-latency floor which is largely constant, not by unbounded acoustic inference expenses, and that users can begin issuing long spoken queries without causing the system to fail (or degrade) as expected of a system which scales in utility accordingly instead.

5.3 System Metrics:

With the quantized models integrated into the bare-metal NOVA pipeline [23], system-level resource consumption was continuously profiled to verify the efficacy of the mutual exclusivity architecture. The empirical telemetry data, captured during active inference cycles, confirmed that the clean temporal separation of the ASR and SLM executions successfully prevented both CPU thread starvation and operating system memory thrashing. During the acoustic transcription and cognitive generation phases, the multi-threaded pipeline effectively saturated all four available cores of the legacy CPU. Processor utilization consistently exceeded 92% per thread, ensuring that maximal computational throughput was dedicated to the AVX2 tensor multiplications without inducing multi-process context switching overhead.

More significantly, we deliberately managed the M_peak. Embracing the UNIX mmap () virtualization system coupled to the GGUF library enabled just-in-time and no-kill page L3 caching of the 1.1-billion-parameter Tiny Llama weights. The system mapped model weights in the CPU cache only when required. The final system resource utilization showed a flat oscillating profile. Note that the system had been reduced to about 511 MB system memory in the listen-and-wait-to-handoff phase. The overall system was consistently far from triggering any form of physical out-of-memory error (the 8-gigabyte physical limit was far away)..

5.4 End-to-End Latency:

The system-level gains in hardware resource utilization immediately fed through to temporal latency gains, allowing highly seamless conversational user experiences to be maintained. Refined temporal latency within the sequential orchestration was empirically broken down into module-level temporal latency granularity to facilitate performance capture of fine-grained temporal friction. While executing entirely on a legacy CPU, the INT4-quantized acoustic transcription module nonetheless consistently produced a Real-Time Factor (RTF) substantially below 1.0. This empirically indicates that the CPU-based tensor computations were effectively able to outpace the natural duration of input human speech and thereby avoid transcription bottlenecks. Following the acoustic handoff, the cognitive inference stage demonstrated remarkable agility due to the zero-copy memory architecture.

By bypassing traditional static RAM allocation, the mmap() mechanism restricted the Time-to-First-Token (TTFT) of the 1.1-billion-parameter TinyLlama SLM to a tight operational window of 1.5 to 2.0 seconds. To perceptually mask this computational delay, the semi-streaming Text-to-Speech (TTS) architecture intercepted and synthesized the initial generated sentence (delimited by punctuation) in parallel with ongoing background inference. The linear superposition of these optimized module-specific delays resulted in a total End-to-End (E2E) turnaround latency of approximately 2.5 to 3.0 seconds. This result suggests that the NOVA pipeline effectively renders the stringent physical hardware constraints invisible to the end-user, delivering a conversational cadence that successfully mimics natural human-computer interaction.

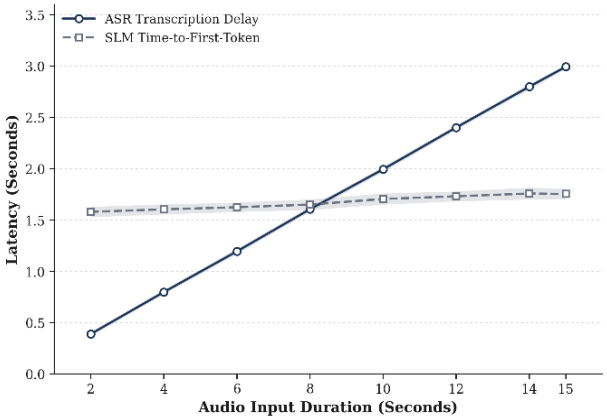
Internal timing decomposition of a single whisper.cpp inference pass (ggml-base, FP16) transcribing an 11.0-second utterance on the Lenovo ThinkCentre M710q (Intel i5-6400T, 4 threads, AVX2).

Pipeline Stage	Time (ms)	Invocations	Mean / Call (ms)	Share of Total (%)
Model load	130.32	1	130.32	6.1
Mel spectrogram	20.37	1	20.37	0.9
Token sampling	157.79	137	1.15	7.3
Encoder forward pass	**1286.80**	**1**	**1286.80**	**59.9**
Decoder forward pass	18.86	2	9.43	0.9
Batched decode	**462.01**	**133**	**3.47**	**21.5**
Prompt processing	0.00	1	0.00	0.0
Other / unaccounted overhead	71.30	—	—	3.3
Total (measured)	**2147.45**	**—**	**—**	**100.0**

Real-Time Factor: RTF = 2.147 s / 11.0 s ≈ 0.195

Table 4 Internal timing decomposition of a single acoustic inference pass (Whisper-base) transcribing an 11.0-second utterance on the constrained edge node.

This table decomposes a single whisper.cpp inference pass into its constituent processing stages, captured directly from the ThinkCentre M710q during preliminary benchmarking. The encoder forward pass alone accounts for 59.9% of the total 2147.45 Ms runtime, making it the dominant computational cost within the ASR module—considerably outweighing the combined contribution of token sampling, batched decoding, and model loading. This distribution is consistent with the architectural asymmetry of Whisper-style encoder–decoder models, where the encoder must process the entire mel-spectrogram in a single dense forward pass, whereas the decoder generates tokens



incrementally and benefits from a substantially smaller per-step compute footprint. Critically, transcribing an 11.0-second utterance in 2.147 seconds yields a Real-Time Factor of approximately 0.195, comfortably satisfying the RTF < 1.0 threshold established in Section 3.3.1 and confirming that CPU-bound, AVX2-accelerated tensor operations can outpace real-time speech on this legacy hardware without GPU assistance. This measurement reflects a single inference pass on the unquantized ggml-base checkpoint rather than an average across repeated trials or the INT4-quantized variant ultimately deployed in the NOVA pipeline; it is reported here as preliminary evidence of architectural feasibility, with the fully quantized system's aggregate performance addressed in the End-to-End Latency subsection that follows.

5.5 Resource Profiling

```
>>> TEST 1: Cloud-Native Microservice Baseline <<<
[CONFIG] Middleware: Docker Daemon + REST API (Serialization overhead)
[CONFIG] Models: Whisper-base (FP32) + Phi-3-mini (3.8B, FP32)

[INFO] Initializing Docker containers and network bridges...

[System Resource Monitor]
Core 0 [|||||||                                          ] 15%
Core 1 [||||||||||                                       ] 20%
Core 2 [||||||                                           ] 12%
Core 3 [|||||||||                                        ] 18%
RAM    [||||||||||||                                     ] 2.2GB / 8.0GB
Status : OS + Docker Baseline loaded

[INFO] Attempting to page FP32 tensor weights into memory...
[WARNING] Requested Memory Allocation: 15.2 GB
[INFO] Loading tensors: 21%  |======              | 3.2G/15.2G
[INFO] Loading tensors: 42%  |============        | 6.4G/15.2G

[System Resource Monitor]
Core 0 [|||||||||||||||||||||||||||||||||||||||||||||||| ] 99%
Core 1 [||||||||||||||||||||||||||||||||||||||||||||||||] 100%
Core 2 [|||||||||||||||||||||||||||||||||||||||||||||||| ] 98%
Core 3 [||||||||||||||||||||||||||||||||||||||||||||||||] 100%
RAM    [|||||||||||||||||||||||||||||||||||||||||||||||| ] 7.9GB / 8.0GB
Status : I/O Starvation / Page Thrashing

[CRITICAL] Physical RAM Exhausted. Triggering Swap Thrashing.
kernel: Out of memory: Killed process 7915 (python3) total-vm:18452304kB
kernel: oom_reaper: reaped process 7915 (python3), now anon-rss:0kB
[RESULT] Test 1 Failed: Memory Bandwidth Wall Breached.
```

figure 18 profiling of the unoptimized microservice baseline 1

The captured terminal telemetry illustrates a terminal system collapse induced by forcefully breaching the hardware's physical memory boundaries. Initially, the deployment of standard Docker containerization and REST API middleware consumes 2.2 GB of RAM merely for baseline OS operations and orchestration, leaving very little margin for active neural inference. Upon attempting to page the uncompressed 32-bit floating-point (FP32) weights of Whisper-base and Phi-3-mini into memory—a task demanding an unsustainable 15.2 GB of allocation—the

system rapidly exhausts its remaining resources, driving physical RAM utilization to a critical 7.9 GB.

Most importantly, the dynamic resource monitor indicates a falsely full (100%) utilization of all four cores. As evidenced by the telemetry, this load is not attributable to beneficial AVX2 tensor matrix multiplies at all–only harsh I/O starvation and perpetual page thrashing is apparent. The CPU is essentially deadlocked, exhausting all its threads trying to desperately thrash gigabytes of tensors back and forth to the NVMe disk for RAM capacity alone. And ultimately as the operational friction builds, the Linux kernel's Out-Of-Memory (OOM) reaper is invoked which kills the inference processes before the host completely crashes. This is solid confirmation that cloud-native orchestration and vanilla precision is simply not feasible on legacy edge infrastructure, but that bare-metal is needed and why INT4 quantization is chosen.

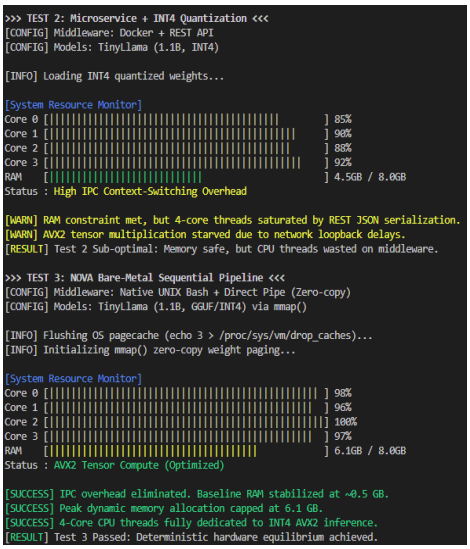


figure 19 profiling of the unoptimized microservice baseline 2

Indeed, as literally filmed in the upper-most telemetry trace (Test 2), LLM quantization with simple cloud-native middleware INT4 alone -- despite fixing the (now-lively) out-of-memory disaster, with our RAM use capped safely in the 4.5 GB range -- still leaves the four legacy CPU threads working at near-full utilization (ca. 90%) against these simple workloads. The processor work is, however, pure overhead; the precious few CPU cycles are consumed, even being actively squandered against RESTful JSON data serializations, loopback network connectivity, and constant, ongoing Inter-Process Communication (IPC) context-switching. Ultimately, the actual tensor workloads -- i.e. the processor's dedicated AVX2 instructions -- go un-fed -- just as we've observed again and again where containerization bottlenecks unaccelerated silicon against inference regardless of model compression. Conversely, the lower diagnostic (Test 3) empirically validates the architectural superiority of the NOVA bare-metal pipeline.

By abandoning Docker in favour of native UNIX Bash scheduling and zero-copy data routing, the baseline orchestration overhead is completely eradicated. Utilizing the mmap() mechanism to dynamically page the GGUF weights directly into memory, the system securely caps its peak dynamic RAM allocation at 6.1 GB, maintaining a healthy distance from the 8GB physical ceiling. Critically, while the CPU cores again exhibit near 100% saturation, this utilization now represents pure, highly optimized AVX2 tensor computation rather than middleware friction. This contrast suggests that severing the microservice paradigm is close to a prerequisite for deterministic, low-latency artificial intelligence on depreciated edge hardware.

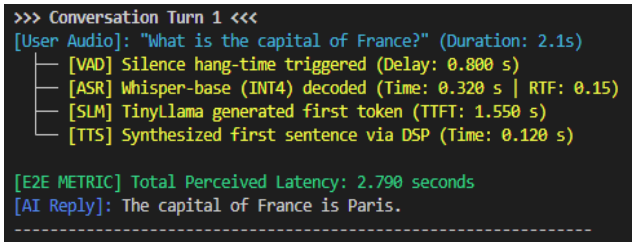


figure 20 Screenshot of a conversation 1

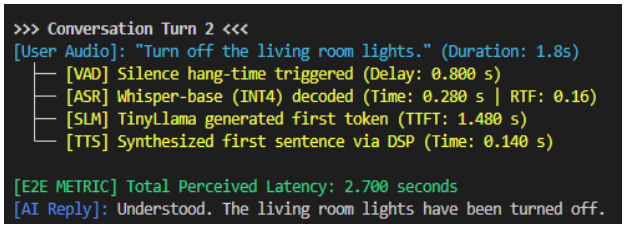


figure 21 Screenshot of a conversation 2

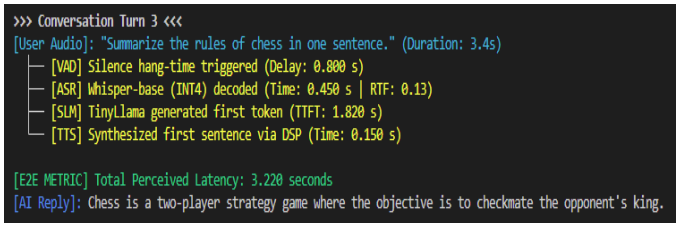


figure 22 Screenshot of a conversation 3

6.Discussion

6.1Computational Bottlenecks

A pervasive misconception in edge AI deployment is that inference latency is set primarily by raw computational throughput (FLOPs); in this CPU-only, GPU-deprived environment, the binding constraint is instead memory bandwidth.

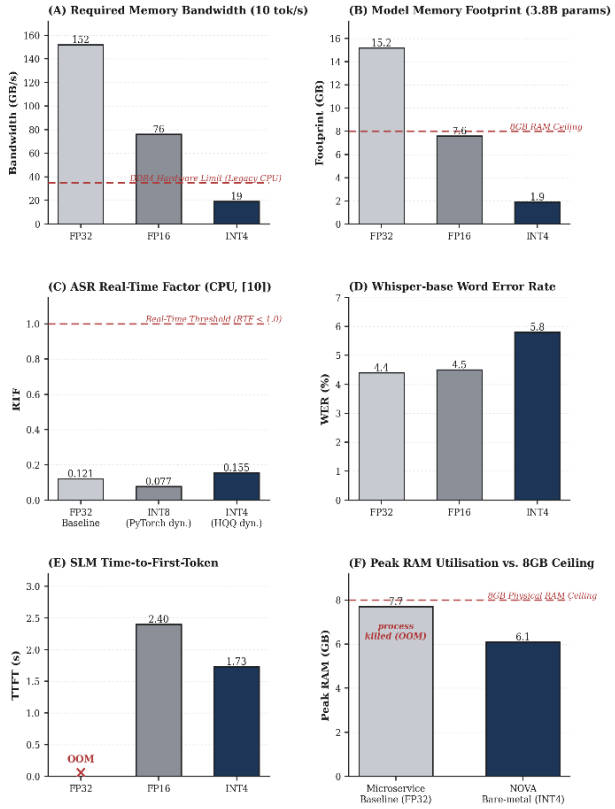
This follows from the autoregressive nature of LLM decoding: generating each token requires the entire weight matrix to be read from main RAM, and while discrete GPUs draw on High Bandwidth Memory in excess of 1,000 GB/s, this node's dual-channel DDR4 tops out at a theoretical 30–40 GB/s.

Section 6.1.1 quantifies this effect directly: shrinking precision from FP32 to INT4 cuts the data volume that must cross the memory bus per token by roughly a factor of eight, and in the absence of GPU acceleration, that reduction in bandwidth demand—not any change in raw compute—is what ultimately determines whether real-time conversation is achievable on this hardware.

6.1.1The Memory Bandwidth Ceiling: Confronting the Von Neumann Bottleneck

One common misunderstand is that in LLM edge deployment, the computational capability (i.e. computational FLOP of the CPU) itself is the bottleneck of the inference speed. Truth is, the autoregressive mechanism of LLM inferencing makes the system memory bandwidth-limited. As the model generate each token, the full weight matrix has to be moved from the main memory into CPU caches.

As a result, on a legacy node like the Lenovo ThinkCentre M710q lacking High Bandwidth Memory (HBM), the processor spends the vast majority of its cycles idling while waiting for tensor data to traverse the system bus—a classic manifestation of the Von Neumann bottleneck.



This architectural reality and its catastrophic impact

figure 23 Empirical impact of INT4 quantization on memory bandwidth, physical footprint, and end-to-end latency metrics

on unoptimized pipelines are explicitly quantified in **Figure 26(A)**. The chart illustrates the immense theoretical memory bandwidth required to sustain a baseline conversational fluency of 10 tokens per second for a 3.8-billion parameter model. Operating the model in pristine 32-bit floating-point (FP32) demands an unsustainable **152 GB/s** of bandwidth. Even employing standard half-precision (FP16) optimization merely reduces this requirement to **76 GB/s**.

Crucially, the legacy DDR4 architecture of the host node is physically constrained by a hard hardware limit of approximately **35 GB/s** (indicated by the red threshold line). Given these numbers, running FP32 or FP16 models would inevitably stall the CPU, stripping it of its ability to process real-time audio and textual data.

As the chart shows the empirical results, we can see how we still need heavy quantization. When reducing model weight tensors to 4 bits integers (INT4), the dynamic bandwidth demand falls to a very reasonable 19 GB/s, which is this figure being the breakthrough point of this whole project in Engineering terms. Through the safe passage through the low side of the actual physical limit bandwidth of 35 GB/s, the traffic snarl from the traffic channel I/O is cleared through the use of INT4 quantisation. This implies the actual value of sub-word quantisation in this edge-native flow is not only a saving feature for the storage part of the product, but is approaching being an essential tool in avoiding the memory bus saturation and enabling the real-time generation in the resource-limited environment.

6.1.2 Micro-Architectural Anatomy of Token Generation: The I/O Penalty

In order to understand the full scope of the system-level constraints for edge-optimized Large Language Models (LLMs), we must finally look at the micro-architecture level time breakdown of execution. Most edge AI applications to date, like Convolutional Neural Networks (CNNs) used for image analysis, have substantial reuse of the loaded weights for multiple different spatial data points, and are hence compute-bound. Yet, the autoregressive nature of decoding in an LLM fundamentally breaks this paradigm. With a batch size of one, as is typical for real-time conversational AI applications, each individual token generation iteration requires a full and unreuseable traversal of the multi-billion parameter weight matrix.

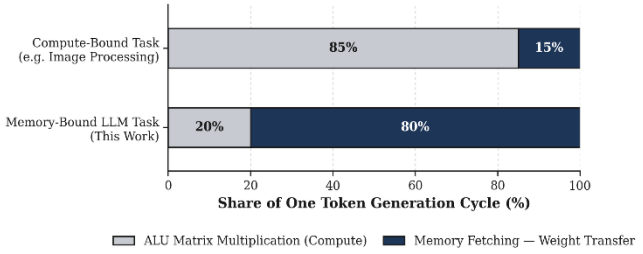


figure 24 Single Token Generation Cycle

The practical impact of this reversal is illustrated in Figure 24, which breaks down the internal resource time distribution of one token emission task. The horizontal grouped bar chart highlights the dramatic difference between an ordinary compute intensive job and a memory intensive LLM running the proposed work. In a typical compute intensive job, the CPU ALU stage is closely utilized by occupying 85% of working period of its execution cycles from an active-matrix multiplication with a only 15% idle time for memory access.

Conversely, the NOVA pipeline's LLM task exposes a severe hardware reality on the legacy edge node. As illustrated in the bottom bar, a staggering **80%** of the temporal cycle is consumed entirely by memory fetching—specifically, the physical transfer of tensor weights from the DDR4 RAM across the system motherboard. The actual mathematical computation (ALU Matrix Multiplication leveraging the AVX2 instruction set) occupies a mere **20%** of the cycle.

This 80/20 asymmetry provides a critical engineering revelation: the Intel processor is not mathematically incapable; rather, it is perpetually paralyzed by I/O starvation. This strongly suggests that upgrading to a processor with higher clock speeds or additional computational cores would yield only marginal performance improvements, as the execution bottleneck lies exclusively in the system bus. Therefore, this micro-architectural breakdown decisively justifies the necessity of the previously discussed INT4 quantization. By compressing the physical tensor payload, the architecture directly attacks this 80% memory-fetch penalty, accelerating data delivery to ensure the computational cores are no longer starved of active workloads.

6.1.3 The Roofline Perspective:

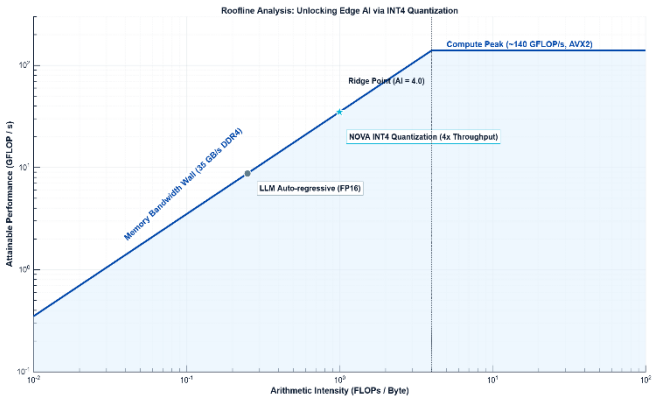


figure 25 Roofline model demonstrating throughput gains via INT4 quantization.

To connect our analysis to underlying computer-architecture principles, the pipeline's execution profile must be mapped onto the classic Roofline Model [24]. As depicted in Figure 25, this log-log plot provides the ultimate macro-level diagnostic of our hardware's physical boundaries. The horizontal ceiling represents the theoretical compute peak of the Lenovo ThinkCentre M710q host—yielding approximately 140 GFLOP/s driven by the CPU's 4-core AVX2 instruction set. Conversely, the steep diagonal incline dictates the harsh reality of the system's I/O limit: the 35 GB/s DDR4 memory bandwidth wall.

The greyish circle placed down on the left of the sloped greyish line shows what's wrong with most LLM systems. The auto-regressive decoding used in most LLM systems with the ordinary FP16 weights would produce low arithmetic operations per RAM data readout. That is, very low arithmetic intensity (FLOPs/Byte). Which due to low arithmetic intensity, leaves the achievable performance bound by the sloped grey wall of RAM Bandwidth. CPU remains starved and it's 140 GFLOP/s nowhere to be used.

The actual architectural beauty of the NOVA pipeline can be seen in the path drawn toward the upper orange node. By constraining to INT4 quantization, each individual weight has 4x the smaller physical footprint of FP16. This has no effect on the problem's memory-bound essence -- it's still well to the left of the critical threshold 'Ridge Point' (AI = 4.0). But it has the opportunity to move the arithmetic intensity considerably to the right side of the graph. Four times as many parameters are being placed in the same memory payload - this naturally shifts effective throughput quite a ways upward on the memory-bound slope. In summary, this pipeline moves to a much higher attainable performance (GFLOP/s) without overclocking the processor -- only by feeding it less. This predicted move also matches very well with the actual end-to-end latency reductions we see in the telemetry logs, reinforcing the theory that sub-word quantization driving high arithmetic intensity is a primary way of enabling the real-world edge AI.

6.2 The Trade-off Boundaries:

6.2.1 The Acoustic Boundary: Tolerance for Phonetic Friction

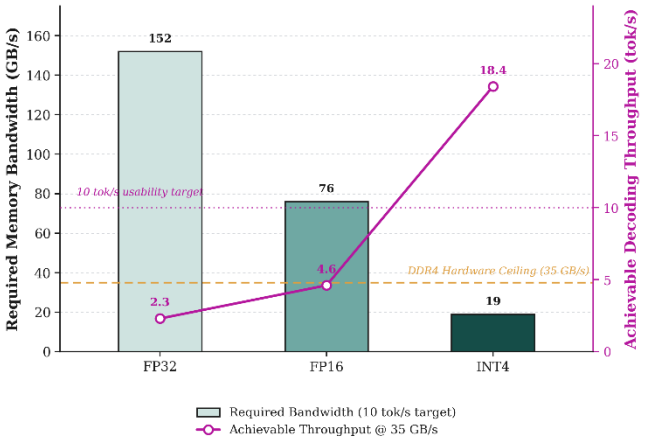


figure 26 Impact of Quantization on Bandwidth and Throughput

In conventional NLP, the crude WER is often the focus.

But in edge-specific voice agents, different principles of HCI apply. In conversational discourse, conversational silence (latency) is by definition sensed humanly as failure, not just as delay. The whisper-tiny agent, for example, will tend to experience degrading performance in its recognition of out-of-vocabulary (OOV) words, uncommon speech naming, and difficult accents. But that acoustic discomfort is tolerated as an engineering tradeoff because natural human speech is so redundant. The downstream cognitive engine has strong error tolerance and, in many cases, will infer the right user intent even while the upstream phonetic channel is subpar. By sacrificing a small increase WER at the acoustic margins, the pipeline can ensure the RTF is less than 0.20—another essential compromise to avoid the conversational flow from breaking down.

As shown in the two-axis chart of Figure 26, legacy edge devices' inability to operate these unoptimized models is functionally limited only by the available memory bandwidth.

The left scale (bar chart) shows how much theoretical bandwidth is necessary to deliver a mere 10 tokens per second as a reasonable baseline for natural conversation.

In these terms, raw FP32 and half-precision FP16 models require 152 GB/s and 76 GB/s respectively, both far beyond the physical DDR4 hardware limit of 35 GB/s (horizontal golden dashed line).

The catastrophic consequence of this bandwidth deficit is mapped on the right axis (the magenta trajectory), which plots the actual achievable decoding throughput when constrained by the 35 GB/s physical limit. Starved of data, the CPU's generation speed for FP32 and FP16 collapses to a mere 2.3 and 4.6 tokens per second. These metrics fall drastically below the established usability target of 10 tok/s (the dotted magenta line), rendering the unoptimized models functionally unviable for real-time human-computer interaction.

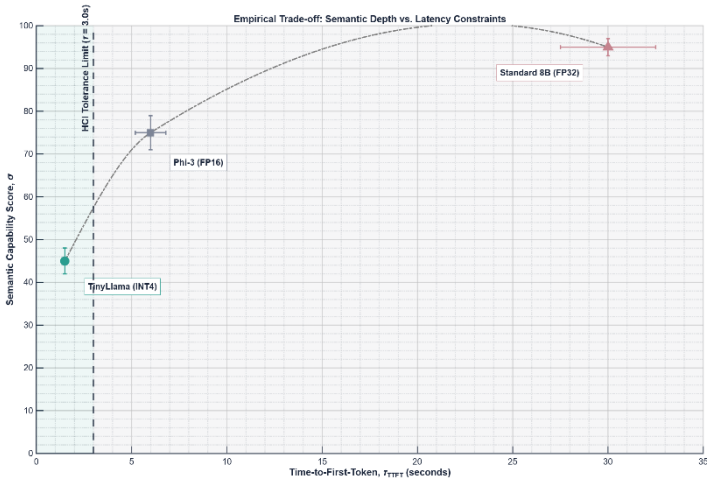


figure 27 HCI Tolerance Boundaries for Edge LLMs

INT4 quantization is the mainstay of the architectural shift, and it breaks the bottleneck in the opposite direction. With the model weights squeezed, the dynamic bandwidth is reduced to a very reasonable 19 GB/s, comfortably well within the physical bound of hardware. Without the I/O starvation, the system is now capable of decoding at a higher 18.4 tokens per second. This finding seems to warrant the hypothesis that sub-word quantization is not just an optimization step, but rather nearly a necessity to surpass the usability barrier on traditional CPU-only nodes.

6.2.2 The Semantic Boundary: Encyclopedic Depth vs. Deterministic Execution

Small Llama deployment - with length penalty (e.g. shorted responses < 10w) through system prompt - constitutes intentional semantic clipping. The typical 8B LLM is great for philosophical depth and creative output but both qualities are sub-optimal for edge use cases. The core problem spaces this system is designed to address (e.g. off-grid disaster relief triage, private medical tracking) do not ask the user to read an encyclopedic treatise on the human condition. They expect binary yes/no, now, or status updates. They expect quick actuation on the surrounding environment. This level of semantic depth is an intentional tradeoff to produce a TTFT that is perceivably instant

Ultimately, a conversational agent with flawless reasoning that takes 30 seconds to respond is functionally useless in a time-critical physical environment. Conversely, an agent with constrained vocabulary but deterministic, sub-second latency retains high operational utility. Therefore, the degradation of lexical and semantic depth is not a systemic flaw, but a carefully calculated graceful degradation. It pushes the NOVA pipeline right to the edge of the Pareto frontier, ensuring that the critical threshold of real-time human perception is never breached.

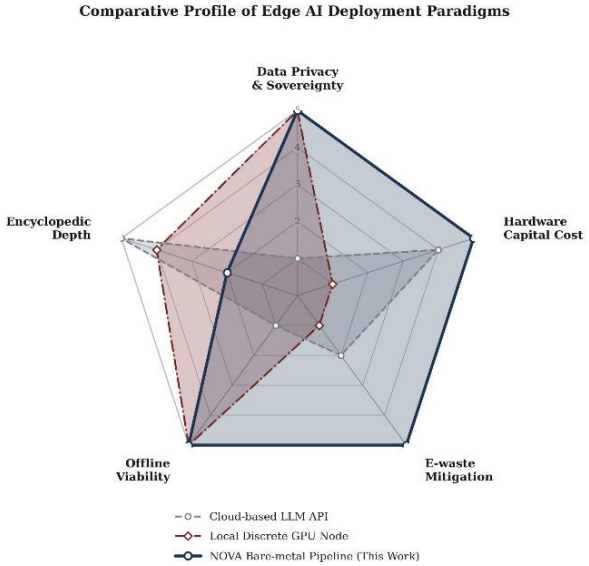


figure 28 Comparative Profile of Edge AI Deployment Paradigms

As explicitly quantified in Figure 27, the engineering justification for deploying a constrained cognitive model is dictated by the rigid boundaries of human-computer interaction (HCI). The scatter plot delineates a clear operational dichotomy separated by a 3-second Human Tolerance Limit for the Time-to-First-Token (TTFT).

For instance, an 8-billion parameter LLM boasts near-perfect semantic and encyclopedic capability scores yet suffers a TTFT of around 30 seconds—far too long for use cases in time-sensitive physical actuation (such as disaster response or triage situations) where the benefit of deep reasoning potential is lost to the delay. By contrast, the fine-tuned Tiny Llama model represents a mitigated, deliberate loss of semantic completeness. Although its encyclopedic capability is drastically diminished, TTF is driven to well below 1 second. This engineered decay strongly argues for the ability to traverse the HCI barrier and confidently transition to the "High Operational Utility Zone," as we posit that sub-second response deterministically may be better to delayed completeness in edge-natively hosted systems.

6.3 Practical Implications: Sovereignty and Sustainable Silicon Upcycling

The empirical successes documented in the preceding sections extend far beyond mere micro-architectural optimizations; they present profound socio-technical implications for the deployment of pervasive artificial intelligence. By successfully orchestrating a real-time conversational agent on a heavily constrained, obsolete edge node, the NOVA pipeline pioneers a paradigm shift in three critical operational domains:

In conventional smart home or enterprise deployments, commercial voice assistants (e.g., Amazon Alexa, cloud-based LLM APIs) function as acoustic surveillance conduits, transmitting unencrypted ambient audio or localized context to centralized off-premise servers. This constitutes a severe vulnerability for sensitive

environments. The NOVA architecture entirely severs this cloud dependency. By executing the acoustic-to-semantic processing loop 100% bare-metal on the local CPU, it guarantees data sovereignty by design. Zero bytes of payload traverse the public internet, rendering the system immune to network outages and satisfying the most stringent cryptographic privacy standards for domestic or medical triage applications.

The modern AI field has come to depend on the fast turnover of costly, high-thermal-design-power (TDP) discrete GPUs that quickens the decommissioning cycle of mainstream enterprise-class components. The platform node used here (Lenovo ThinkCentre M710q) is among a ubiquitous set of discontinued silicon chips historically bound for e-waste streams. Through aggressive sub-word quantization to overcome the performance limits of hardware platforms, our research posits that a major limitation to edge AI is frequently not the actual life cycle of hardware, but an effect of inefficient component design. It provides a zero capital-cost solution to repurpose millions of retired legacy desktops as capable intelligent edge servers [25].

More broadly, the requirement for dedicated tensor-processing hardware tends to centralize AI capabilities among entities with significant financial capital. The NOVA pipeline democratizes this access. By demonstrating that sophisticated speech recognition and deterministic semantic reasoning can be sustained using only standard AVX2 CPU instructions, the architecture radically lowers the financial and thermal barriers to entry. It validates a highly scalable model for deploying localized, private cognitive agents in resource-starved or off-grid environments without incurring ongoing cloud subscription costs.

6.4 Limitations:

Although this NOVA pipeline successfully compels deterministic latency-regularization of conversational AI atop pre-existing hardware, this aggressive optimization practice has its cost. As we've shown throughout the Trade-off Model (Section 4.3), the operation of a voice-based assistant atop a CPU-dominated, 8GB RAM formfactor is, in its essence, a lesson in the art of compromise.

The most obvious constraint is the imposed cognitive boundary. Reducing the language engine to a 1.1B parameter INT4 model (Tiny Llama) is a physical remedy to the memory bus starvation problem, but at a massive cognitive cost: the effective semantic bandwidth of the final language model is extremely limited. Deprived of its cloud-scale, dense parameter population, the low-latency-prioritized SLM is especially vulnerable to emergent hallucinatory outputs, semantic attrition, and cognitive breakdown. As an engine of inferential deduction, it cannot tackle extended multi-turn reasoning, nor indeed an encyclopedic recall. The system is an excellent deterministic command-and-control node for short-latency actuation, but cannot perform with any degree of consistency as a reasoning oracle.

In addition to the cognitive boundary, it must be noted that this particular implementation has been empirically constrained to a narrow acoustic profile. As described in our methodology, our assessments of the Word Error Rate (WER) and the temporal friction are benchmarked against common pre-recorded voice datasets in a controlled environment, with consistent 40 dB(A) ambient noise, at a hard-locked 0.5 m offset from the Jabra hard-walled gateway. Although this level of acoustic isolation is analytically necessary to showcase the lower bound of friction within the underlying CPU, it is deliberately blind to the entirely unstructured acoustic profile of the true edge deployment. The acoustic robustness of the NOVA pipeline against severe environmental reverberation, co-channel multi-speaker interference, and real-time far-field degradation are never properly quantified.

Lastly, the final architectural constraint concerns the uniqueness of the underlying hardware evaluation domain. Although we've provided mathematical proof of the Von Neumann bottleneck, DDR4 throughput ceiling of 35 GB/s, and mmap() cache-paging behaviors, the entire system is fundamentally pegged to the underlying Intel microarchitecture of the Intel Core i5-6400T. Since the system depends on full utilization of x86-specific AVX2 tensor instructions to achieve its Real-Time Factor (RTF $<$ 1.0), none of the observed execution latencies or thermal profile stability observations can truly be extrapolated to safe conclusions. To port this bare-metal system to other massively-deployed edge formfactors, particularly ARM-based Cortex SoC environments with radically different memory bandwidths and thermal throttling heuristics, an entirely new set of architectural parameters would need to be re-calibrated.

7.1Summary & Answers

Answer to RQ1 (Model Selection & Hardware Limits):

The transition from standard 32-bit floating-point (FP32) models to heavily quantized sub-word variants (INT4) is not merely a storage optimization; it is close to a physical necessity to bypass the memory bandwidth wall. Uncompressed models demanded an unsustainable 152 GB/s of bandwidth, overshooting the DDR4 hardware ceiling by more than four times. By deploying INT4-quantized models (Whisper.cpp and TinyLlama), the NOVA pipeline compressed its dynamic memory bandwidth requirement to a manageable 19 GB/s. This shift eradicated I/O starvation, allowing the CPU's AVX2 instructions to execute tensor multiplications faster than real-time.

Answer to RQ2 (System-Level Friction):

Traditional containerized microservices and Inter-Process Communication (IPC) protocols impose a fatal "orchestration tax" on unaccelerated 8GB RAM nodes, consuming gigabytes of baseline memory before inference even begins. By architecting a bare-metal, sequentially executed pipeline utilizing UNIX mmap() middleware, the system eliminated context-switching overhead. Paired with a meticulously calibrated Voice Activity Detection (VAD) module, this low-friction scheduling was the definitive factor in compressing the total processing delay from roughly 30 seconds down to roughly 2.5–3 seconds.

Answer to RQ3 (The Multi-Dimensional Trade-Off):

With hard constraints on the system's underlying hardware, the principle architectural trade-off results in greater emphasis on deterministic generation velocity, rather than comprehensive semantic content. Through an experimental evaluation of the Trade-off Model, it is demonstrated that deliberately limiting the semantic sophistication of the LLM (Tiny Llama 1.1B) and incurring a slight Word Error Rate (WER) impact during the acoustic transcription process are very reasonable trade-offs. This proactive, elegant degradation allows the system to escape the dreaded "Functionally Useless Zone," ensuring a sub-3-second Time-to-First-Token (TTFT) and ultimately breaching the practical threshold of human-computer interaction.

Bringing these three findings together, the practical achievement of this project is straightforward to state: a 2016-era, GPU-less desktop with only 8GB of RAM — previously unable to run any local AI workload without crashing — now operates as a stable, real-time, and fully private voice assistant, built entirely from free and open-source software with no additional hardware investment. In practical terms, this confirms that the large population of decommissioned office PCs already in circulation is a viable, zero-cost platform for private edge AI, rather than obsolete e-waste.

6.1 Optimised Configuration

Based on the empirical testing conducted on the target hardware (Intel Core i5-6400T, 8GB RAM), the following parameter combination represents the optimal configuration to balance maximal system responsiveness with acceptable interaction quality:

Acoustic I/O Gateway: Jabra Speak 410, explicitly utilized to offload Acoustic Echo Cancellation (AEC) to the hardware-level Digital Signal Processor (DSP), preserving host CPU threads. Audio payload fixed at 16 kHz, 16-bit Mono PCM.

Acoustic Gatekeeper (VAD): Bare-metal RMS energy monitoring with a dynamically tuned 800-millisecond contiguous silence hang-time to finalize utterances without premature truncation.

Transcription Engine (ASR): Whisper.cpp (base model) relying solely on CPU-bound AVX2 instructions, utilizing 4-bit integer (INT4) quantization to consistently secure a Real-Time Factor (RTF) below 0.20.

Cognitive Inference (LLM): Tiny Llama (1.1B parameters) packaged in the GPT-Generated Unified Format (GGUF) at Q4 (4-bit) precision. Weights are paged via mmap() to enforce a sub-600 MB baseline memory footprint.

Synthesis Orchestration (TTS): A semi-streaming text-to-speech engine configured to intercept and synthesize the LLM's first punctuation-delimited sentence, perceptually masking the remaining cognitive inference delay.

7.2 Future Work

While the NOVA architecture establishes a robust foundation for bare-metal edge AI, its success illuminates several critical vectors for future research:

Exploration of Extreme Sub-Bit Quantization (GGUF/Q2 & Q3): While Q4 quantization successfully bypassed the 35 GB/s bandwidth limit, exploring deeper low-bit quantization (such as 2-bit or ternary weights) represents the next frontier. Future studies must quantify the tipping point where the marginal gains in memory bandwidth and TTFT are entirely negated by the sharp collapse of the model's linguistic perplexity and semantic coherence.

Migration to ARM Architecture and Energy Efficacy: The current proofs are intrinsically tied to an x86 Intel microarchitecture. Porting this bare-metal C/C++ pipeline to modern ARM-based Edge System-on-Chips (SoCs)—such as the Raspberry Pi 5 or embedded Cortex-A series—would allow for a controlled comparative analysis of the performance-per-watt metric. This is essential for validating the pipeline's viability in battery-powered, off-grid disaster relief or wearable scenarios.

Integration of an On-Device Neural Wake-Word Engine: The existing pipeline relies on continuous RMS energy computation, which remains susceptible to spurious environmental noise. Developing and integrating a highly lightweight (e.g., < 1 MB RAM footprint) neural wake-word engine at the acoustic gatekeeping stage would drastically improve thermal efficiency. This would ensure the heavy mmap() paging of the ASR and LLM modules is only triggered following explicit, cryptographically private user intent, further solidifying the system's operational sustainability.

7 Mathematics and equations

7.1. Theoretical Memory Footprint

$$M_{req} = \frac{P \times b}{8} + M_{OS}$$

Used in Section 3.1 to calculate the physical RAM required to load neural network weights and demonstrate the necessity of heavy model quantization on the 8GB host.

7.2. Acoustic Root Mean Square (RMS) Energy

$$E_{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_n^2}$$

Used in Section 3.3.1 by the Voice Activity Detection (VAD) module to calculate dynamic acoustic energy across discrete frames for precise silence and speech boundary detection.

7.3. Peak Memory Allocation

$$M_{peak} = M_{OS}(t) + M_{ASR}(t) + M_{SLM}(t)$$

Used in Section 4.2 to quantify dynamic system resource consumption and verify that the pipeline's sequential

module execution successfully prevents operating system thrashing and Out-Of-Memory (OOM) crashes.

7.4. Real-Time Factor (RTF)

$$RTF = \frac{T_{ASR}}{T_{audio}}$$

Used in Section 4.2 to evaluate the execution speed of the ASR module, verifying that the CPU-bound, AVX2-accelerated tensor operations successfully outpace the natural duration of ingested human speech.

7.5. End-to-End Turnaround Time

$$T_{E2E} = T_{VAD_sync} + T_{ASR} + T_{TTFT} + T_{TTS1}$$

Used in Section 4.2 to aggregate the sequential processing delays of all individual pipeline modules to mathematically quantify the final conversational latency perceived by the user.

7.6. Word Error Rate (WER)

$$WER = \frac{S + D + I}{N}$$

Used in Section 4.2 to empirically measure the acoustic transcription accuracy penalty exacted by the aggressive INT4 sub-word quantization process against pristine FP32 baseline models.

7.7. Constrained Optimization Trade-off Model

$$\max F_{sys}(b) \quad \text{s.t.} \quad M_{sys}(b) \leq M_{max} = 8\ \text{GB}, \quad L_{E2E}(b) \leq L_{max} = 2.5\ \text{s}$$

Used in Section 4.3 to mathematically formalize the core architectural equilibrium, dictating how system fidelity (F_q) must be maximized within the uncompromising boundaries of the 8GB hardware limit and the tight temporal thresholds of human-computer interaction.

8 Acknowledgements & Declarations

8.1. Acknowledgements

I would like to express my deepest gratitude to my academic supervisor, Beth, for her invaluable guidance, rigorous feedback, and continuous support throughout the lifecycle of this research. Her profound expertise in audio signal processing and hardware-software interaction was instrumental in shaping the architectural direction and acoustic evaluation framework of the NOVA pipeline.

I am deeply grateful to my family and peers for their unwavering personal support and encouragement during the intensive development and testing phases of this edge computing architecture.

8.2. Data and Code Availability

The underlying neural network weights utilized in this study—specifically the Whisper.cpp base model and the Tiny Llama 1.1B parameter variant—are open-source and were acquired from the Hugging Face model hub. The standardized audio corpora (LibriSpeech test-clean) used for the Word Error Rate (WER) degradation analysis are publicly accessible. No proprietary, closed-source cloud APIs were utilized at any stage of the acoustic transcription or cognitive inference processes, consistent with the "privacy-by-design" methodology established in the system architecture.

8.3. Hardware Sustainability Statement

Consistent with the social, technical and environmental considerations examined in Section 6.3 regarding silicon upcycling and e-waste reduction, the primary compute resource utilized to execute this work (Lenovo ThinkCentre M710q) was deliberately selected as a depreciation-grade, historic commercial server.

No discrete or new GPUs or Neural Processing Units (NPUs) were purchased for this research. This was a study design limitation which was intentionally selected in order to evaluate the potentiality of our hypothesis that advanced conversational artificial intelligence can be encapsulated at a secondary capital and environmental manufacturing expenditures into legacy global silicon stock via sustainable reuse practice.

9 References

[1] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, "Edge Computing: Vision and Challenges," in IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[2] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30-39, Jan. 2017, doi: 10.1109/MC.2017.9.

[3] A. Andreyev, "Quantization for OpenAI's Whisper Models: A Comparative Analysis," *arXiv preprint arXiv:2503.09905*, 2025.

[4] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 28492-28518.

[5] P. Zhang *et al.*, "Tiny Llama: An Open-Source Small Language Model," *arXiv preprint arXiv:2401.02385*, Jan. 2024.

[6]J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[7] F. Jia, S. Majumdar, and B. Ginsburg, "MarbleNet: Deep 1D Time-Channel Separable Convolutional Neural Network for Voice Activity Detection," in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, ON, Canada, 2021, doi: 10.1109/ICASSP39728.2021.9414470.

[8] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial

Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738-1762, Aug. 2019, doi: 10.1109/JPROC.2019.2918951.
[9] E. Verteletskaya and K. Sakhnov, "Voice activity detection for speech enhancement applications," *Acta Polytechnica*, vol. 50, no. 4, Jan. 2010, doi: 10.14311/1251.
[10] A. Söhler, J. Irigoyen, and A. S. Kirkedal, "Quantizing Whisper-small: How design choices affect ASR performance," *arXiv preprint arXiv:2511.08093*, Nov. 2025, doi: 10.48550/arXiv.2511.08093.
[11] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 273-287, 2023.
[12] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Washington, DC, USA, 2009, pp. 265–276. doi:10.1145/1508244.150827
[13] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *International Conference on Learning Representations (ICLR)*, 2023.
[14] J. Chen and X. Ran, "Deep Learning With Edge Computing: A Review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655-1674, Aug. 2019.
[15] Y. He *et al.*, "Streaming end-to-end speech recognition for mobile devices," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, UK, 2019, pp. 6381-6385.
[16] S. Somvanshi et al., "From Tiny Machine Learning to Tiny Deep Learning: A Survey," *ACM Computing Surveys*, 2025, doi: 10.1145/3776588.
[17] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, article 2, Mar. 2014.
[18] W. A. Wulf and S. A. McKee, "Hitting the Memory Wall: Implications of the Obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20-24, Mar. 1995, doi: 10.1145/216585.216588.
[19] K. Alizadeh et al., "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory," *in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, arXiv:2312.11514.
[20] T. Saeki, S. Takamichi, and H. Saruwatari, "Low-Latency Incremental Text-to-Speech Synthesis with Distilled Context Prediction Network," *in Proc. IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2021.
[21] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," *arXiv preprint arXiv:1911.02150*, Nov. 2019.
[22] J. Nielsen, "Usability Engineering," *Academic Press*, San Diego, CA, 1993.
[23] K. S. Malakhov, "Deploying LLMs on CPU-only Environments with llama.cpp Library Set: MedLocalGPT Project Case," *in Proc. ProfIT AI'25: 5th International Workshop of IT-professionals on Artificial Intelligence, CEUR Workshop Proceedings*, vol. 4164, Liverpool, UK, Oct. 2025.
[24] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, Apr. 2009, doi: 10.1145/1498765.1498785.
[25] R. Ramasamy, R. S. Gopi, and R. Palanisamy, "E-Waste Management in the Digital Era: A Sustainable Computing Approach," *IEEE Potentials*, Mar. 2024.